10.6 Exercises

For help with exercises, please visit the site, www.wiley.com/college/goodrich.

Reinforcement

R-10.1 Give a concrete implementation of the pop method in the context of the MutableMapping class, relying only on the five primary abstract methods of that class.

R-10.2 Give a concrete implementation of the items( ) method in the context of the MutableMapping class, relying only on the five primary abstract methods of that class. What would its running time be if directly applied to the UnsortedTableMap subclass?

R-10.3 Give a concrete implementation of the items( ) method directly within the UnsortedTableMap class, ensuring that the entire iteration runs in $O(n)$ time.

R-10.4 What is the worst-case running time for inserting n key-value pairs into an initially empty map M that is implemented with the UnsortedTableMap class?

R-10.5 Reimplement the UnsortedTableMap class from Section 10.1.5, using the PositionalList class from Section 7.4 rather than a Python list.

R-10.6 Which of the hash table collision-handling schemes could tolerate a load factor above 1 and which could not?

R-10.7 Our Position classes for lists and trees support the __eq__ method so that two distinct position instances are considered equivalent if they refer to the same underlying node in a structure. For positions to be allowed as keys in a hash table, there must be a definition for the __hash__ method that is consistent with this notion of equivalence. Provide such a __hash__ method.

R-10.8 What would be a good hash code for a vehicle identification number that is a string of numbers and letters of the form "9X9XX99X9XX999999," where a "9" represents a digit and an "X" represents a letter?

R-10.9 Draw the 11-entry hash table that results from using the hash function, $h(i) = (3i+5) \bmod 11$, to hash the keys 12, 44, 13, 88, 23, 94, 11, 39, 20, 16, and 5, assuming collisions are handled by chaining.

R-10.10 What is the result of the previous exercise, assuming collisions are handled by linear probing?

R-10.11 Show the result of Exercise R-10.9, assuming collisions are handled by quadratic probing, up to the point where the method fails.

R-10.12 What is the result of Exercise R-10.9 when collisions are handled by double hashing using the secondary hash function $h'(k) = 7 - (k \bmod 7)$?

R-10.13 What is the worst-case time for putting n entries in an initially empty hash table, with collisions resolved by chaining? What is the best case?

R-10.14 Show the result of rehashing the hash table shown in Figure 10.6 into a table of size 19 using the new hash function $h(k) = 3k \bmod 17$.

R-10.15 Our HashMapBase class maintains a load factor $\lambda \leq 0.5$. Reimplement that class to allow the user to specify the maximum load, and adjust the concrete subclasses accordingly.

R-10.16 Give a pseudo-code description of an insertion into a hash table that uses quadratic probing to resolve collisions, assuming we also use the trick of replacing deleted entries with a special "deactivated entry" object.

R-10.17 Modify our ProbeHashMap to use quadratic probing.

R-10.18 Explain why a hash table is not suited to implement a sorted map.

R-10.19 Describe how a sorted list implemented as a doubly linked list could be used to implement the sorted map ADT.

R-10.20 What is the worst-case asymptotic running time for performing n deletions from a SortedTableMap instance that initially contains $2n$ entries?

R-10.21 Consider the following variant of the _find_index method from Code Fragment 10.8, in the context of the SortedTableMap class:

```
def _find_index(self, k, low, high):
  if high < low:
    return high + 1
  else:
    mid = (low + high) // 2
    if self._table[mid]._key < k:
      return self._find_index(k, mid + 1, high)
    else:
      return self._find_index(k, low, mid - 1)
```

Does this always produce the same result as the original version? Justify your answer.

R-10.22 What is the expected running time of the methods for maintaining a maxima set if we insert n pairs such that each pair has lower cost and performance than one before it? What is contained in the sorted map at the end of this series of operations? What if each pair had a lower cost and higher performance than the one before it?

R-10.23 Draw an example skip list S that results from performing the following series of operations on the skip list shown in Figure 10.13: del S[38], S[48] = 'x', S[24] = 'y', del S[55]. Record your coin flips, as well.

R-10.24 Give a pseudo-code description of the __delitem__ map operation when using a skip list.

R-10.25 Give a concrete implementation of the pop method, in the context of a MutableSet abstract base class, that relies only on the five core set behaviors described in Section 10.5.2.

R-10.26 Give a concrete implementation of the isdisjoint method in the context of the MutableSet abstract base class, relying only on the five primary abstract methods of that class. Your algorithm should run in $O(\min(n,m))$ where n and m denote the respective cardinalities of the two sets.

R-10.27 What abstraction would you use to manage a database of friends' birthdays in order to support efficient queries such as "find all friends whose birthday is today" and "find the friend who will be the next to celebrate a birthday"?

Creativity

C-10.28 On page 406 of Section 10.1.3, we give an implementation of the method setdefault as it might appear in the MutableMapping abstract base class. While that method accomplishes the goal in a general fashion, its efficiency is less than ideal. In particular, when the key is new, there will be a failed search due to the initial use of __getitem__, and then a subsequent insertion via __setitem__. For a concrete implementation, such as the UnsortedTableMap, this is twice the work because a complete scan of the table will take place during the failed __getitem__, and then another complete scan of the table takes place due to the implementation of __setitem__. A better solution is for the UnsortedTableMap class to override setdefault to provide a direct solution that performs a single search. Give such an implementation of UnsortedTableMap.setdefault.

C-10.29 Repeat Exercise C-10.28 for the ProbeHashMap class.

C-10.30 Repeat Exercise C-10.28 for the ChainHashMap class.

C-10.31 For an ideal compression function, the capacity of the bucket array for a hash table should be a prime number. Therefore, we consider the problem of locating a prime number in a range $[M, 2M]$. Implement a method for finding such a prime by using the ***sieve algorithm***. In this algorithm, we allocate a $2M$ cell Boolean array A, such that cell i is associated with the integer i. We then initialize the array cells to all be "true" and we "mark off" all the cells that are multiples of 2, 3, 5, 7, and so on. This process can stop after it reaches a number larger than $\sqrt{2M}$. (Hint: Consider a bootstrapping method for finding the primes up to $\sqrt{2M}$.)

C-10.32 Perform experiments on our ChainHashMap and ProbeHashMap classes to measure its efficiency using random key sets and varying limits on the load factor (see Exercise R-10.15).

C-10.33 Our implementation of separate chaining in ChainHashMap conserves memory by representing empty buckets in the table as None, rather than as empty instances of a secondary structure. Because many of these buckets will hold a single item, a better optimization is to have those slots of the table directly reference the _Item instance, and to reserve use of secondary containers for buckets that have two or more items. Modify our implementation to provide this additional optimization.

C-10.34 Computing a hash code can be expensive, especially for lengthy keys. In our hash table implementations, we compute the hash code when first inserting an item, and recompute each item's hash code each time we resize our table. Python's dict class makes an interesting trade-off. The hash code is computed once, when an item is inserted, and the hash code is stored as an extra field of the item composite, so that it need not be recomputed. Reimplement our HashTableBase class to use such an approach.

C-10.35 Describe how to perform a removal from a hash table that uses linear probing to resolve collisions where we do not use a special marker to represent deleted elements. That is, we must rearrange the contents so that it appears that the removed entry was never inserted in the first place.

C-10.36 The quadratic probing strategy has a clustering problem related to the way it looks for open slots. Namely, when a collision occurs at bucket $h(k)$, it checks buckets $A[(h(k)+i^2) \bmod N]$, for $i = 1,2,\ldots,N-1$.

a. Show that $i^2 \bmod N$ will assume at most $(N+1)/2$ distinct values, for N prime, as i ranges from 1 to $N-1$. As a part of this justification, note that $i^2 \bmod N = (N-i)^2 \bmod N$ for all i.

b. A better strategy is to choose a prime N such that $N \bmod 4 = 3$ and then to check the buckets $A[(h(k) \pm i^2) \bmod N]$ as i ranges from 1 to $(N-1)/2$, alternating between plus and minus. Show that this alternate version is guaranteed to check every bucket in A.

C-10.37 Refactor our ProbeHashMap design so that the sequence of secondary probes for collision resolution can be more easily customized. Demonstrate your new framework by providing separate concrete subclasses for linear probing and quadratic probing.

C-10.38 Design a variation of binary search for performing the multimap operation find_all(k) implemented with a sorted search table that includes duplicates, and show that it runs in time $O(s+\log n)$, where n is the number of elements in the dictionary and s is the number of items with given key k.

C-10.39 Although keys in a map are distinct, the binary search algorithm can be applied in a more general setting in which an array stores possibly duplicative elements in nondecreasing order. Consider the goal of identifying the index of the *leftmost* element with key greater than or equal to given k. Does the _find_index method as given in Code Fragment 10.8 guarantee such a result? Does the _find_index method as given in Exercise R-10.21 guarantee such a result? Justify your answers.

C-10.40 Suppose we are given two sorted search tables S and T, each with n entries (with S and T being implemented with arrays). Describe an $O(\log^2 n)$-time algorithm for finding the kth smallest key in the union of the keys from S and T (assuming no duplicates).

C-10.41 Give an $O(\log n)$-time solution for the previous problem.

C-10.42 Suppose that each row of an $n \times n$ array A consists of 1's and 0's such that, in any row of A, all the 1's come before any 0's in that row. Assuming A is already in memory, describe a method running in $O(n \log n)$ time (not $O(n^2)$ time!) for counting the number of 1's in A.

C-10.43 Given a collection C of n cost-performance pairs (c, p), describe an algorithm for finding the maxima pairs of C in $O(n \log n)$ time.

C-10.44 Show that the methods above(p) and prev(p) are not actually needed to efficiently implement a map using a skip list. That is, we can implement insertions and deletions in a skip list using a strictly top-down, scan-forward approach, without ever using the above or prev methods. (Hint: In the insertion algorithm, first repeatedly flip the coin to determine the level where you should start inserting the new entry.)

C-10.45 Describe how to modify a skip-list representation so that index-based operations, such as retrieving the item at index j, can be performed in $O(\log n)$ expected time.

C-10.46 For sets S and T, the syntax S ^ T returns a new set that is the symmetric difference, that is, a set of elements that are in precisely one of S or T. This syntax is supported by the special __xor__ method. Provide an implementation of that method in the context of the MutableSet abstract base class, relying only on the five primary abstract methods of that class.

C-10.47 In the context of the MutableSet abstract base class, describe a concrete implementation of the __and__ method, which supports the syntax S & T for computing the intersection of two existing sets.

C-10.48 An ***inverted file*** is a critical data structure for implementing a search engine or the index of a book. Given a document D, which can be viewed as an unordered, numbered list of words, an inverted file is an ordered list of words, L, such that, for each word w in L, we store the indices of the places in D where w appears. Design an efficient algorithm for constructing L from D.

C-10.49 Python's collections module provides an OrderedDict class that is unrelated to our sorted map abstraction. An OrderedDict is a subclass of the standard hash-based dict class that retains the expected $O(1)$ performance for the primary map operations, but that also guarantees that the `__iter__` method reports items of the map according to first-in, first-out (FIFO) order. That is, the key that has been in the dictionary the longest is reported first. (The order is unaffected when the value for an existing key is overwritten.) Describe an algorithmic approach for achieving such performance.

Projects

P-10.50 Perform a comparative analysis that studies the collision rates for various hash codes for character strings, such as various polynomial hash codes for different values of the parameter a. Use a hash table to determine collisions, but only count collisions where different strings map to the same hash code (not if they map to the same location in this hash table). Test these hash codes on text files found on the Internet.

P-10.51 Perform a comparative analysis as in the previous exercise, but for 10-digit telephone numbers instead of character strings.

P-10.52 Implement an OrderedDict class, as described in Exercise C-10.49, ensuring that the primary map operations run in $O(1)$ expected time.

P-10.53 Design a Python class that implements the skip-list data structure. Use this class to create a complete implementation of the sorted map ADT.

P-10.54 Extend the previous project by providing a graphical animation of the skip-list operations. Visualize how entries move up the skip list during insertions and are linked out of the skip list during removals. Also, in a search operation, visualize the scan-forward and drop-down actions.

P-10.55 Write a spell-checker class that stores a lexicon of words, W, in a Python set, and implements a method, check(s), which performs a ***spell check*** on the string s with respect to the set of words, W. If s is in W, then the call to check(s) returns a list containing only s, as it is assumed to be spelled correctly in this case. If s is not in W, then the call to check(s) returns a list of every word in W that might be a correct spelling of s. Your program should be able to handle all the common ways that s might be a misspelling of a word in W, including swapping adjacent characters in a word, inserting a single character in between two adjacent characters in a word, deleting a single character from a word, and replacing a character in a word with another character. For an extra challenge, consider phonetic substitutions as well.

Chapter Notes

Hashing is a well-studied technique. The reader interested in further study is encouraged to explore the book by Knuth [65], as well as the book by Vitter and Chen [100]. Skip lists were introduced by Pugh [86]. Our analysis of skip lists is a simplification of a presentation given by Motwani and Raghavan [80]. For a more in-depth analysis of skip lists, please see the various research papers on skip lists that have appeared in the data structures literature [59, 81, 84]. Exercise C-10.36 was contributed by James Lee.